

A Comparative Study of GP-based and State-of-the-art Classifiers on a Synthetic Machine Learning Benchmark

Patryk Orzechowski^{*†}
University of Pennsylvania
Philadelphia, PA 19104, USA
patryk.orzechowski@gmail.com

Paweł Renc[‡]
Sano, Centre for Computational
Medicine
Kraków, Poland
p.renc@sanoscience.org

William La Cava
Boston Children's Hospital
Harvard Medical School
Boston, USA
william.lacava@childrens.harvard.edu

Jason H. Moore
Cedars-Sinai Medical Center
Los Angeles, CA, USA
jason.moore@csmc.edu

Arkadiusz Sitek
Sano, Centre for Computational
Medicine
Kraków, Poland
a.sitek@sanoscience.org

Jarosław Wąs
AGH University of Science and
Technology
Kraków, Poland
jarek@agh.edu.pl

Joost Wagenaar
University of Pennsylvania
Philadelphia, PA, USA
joostw@seas.upenn.edu

ABSTRACT

In this paper we compare performance of genetic programming-based symbolic classifiers on a novel synthetic machine learning benchmark called DIGEN. This framework and collection of 40 different classification problems was designed specifically to differentiate performance of leading machine learning methods.

CCS CONCEPTS

• **Information systems** → **Information retrieval**; • **Computing methodologies** → **Cluster analysis**; *Search methodologies*; Bio-inspired approaches; • **Theory of computation** → *Massively parallel algorithms*.

KEYWORDS

benchmarking, genetic programming, machine learning, evolutionary algorithms, classification

ACM Reference Format:

Patryk Orzechowski, Paweł Renc, William La Cava, Jason H. Moore, Arkadiusz Sitek, Jarosław Wąs, and Joost Wagenaar. 2022. A Comparative Study of GP-based and State-of-the-art Classifiers on a Synthetic Machine Learning

Benchmark. In *Genetic and Evolutionary Computation Conference Companion (GECCO '22 Companion)*, July 9–13, 2022, Boston, MA, USA. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3520304.3529056>

1 INTRODUCTION

Classification, alongside with regression, clustering and dimensionality reduction, is one of the most basic and popular tasks in machine learning (ML). Over the years multiple benchmarks have been proposed in order to measure performance of ML methods [2, 9, 13, 17, 20, 21]. The major aim of such benchmarks is to verify how the performance of a given method differs across dozens or hundreds of problems. Although the abundance of available data for testing algorithms should allow us to get a general sense of a methods' performance, *more* doesn't necessarily mean *better*. There are several issues that are apparent in existing collections of datasets. First, collections of problems included in a benchmark may not sufficiently differentiate ML methods. This arises from the inclusion of multiple datasets for which the performances remain the same or nearly the same. Second, multiple datasets come from real world experiments with unknown ideal performance. Real-world tasks are certainly important to study since the end goal of any benchmark is to have a practical application. However, it also brings uncertainty to benchmarking, as their ground truth remains unknown. Third, many benchmarking frameworks have underlying quality issues: they may be non-standardized, poorly documented, difficult to handle, lack transparency, or not include results that could serve as a baseline. This leads to a situation in which there are only a small number of synthetic datasets of high quality which could actually be used for comparing classifiers.

Since its earliest attempts, the use of genetic programming (GP) for classification, especially for multi-class problems, has been considered challenging due to increased computational cost, issues with bloat control, and frequent fitness evaluation [4–7, 27]. Even more recently (2018), the performance of a GP-based symbolic regression

^{*}corresponding author

[†]PO is affiliated with AGH University of Science and Technology, Dept. of Automatics and Robotics, al. Mickiewicza 30, 30-059 Krakow, Poland

[‡]PR is also affiliated with AGH University of Science and Technology, Dept. of Applied Computer Science, al. Mickiewicza 30, 30-059 Krakow, Poland

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

GECCO '22 Companion, July 9–13, 2022, Boston, MA, USA

© 2022 Copyright held by the owner/author(s).

ACM ISBN 978-1-4503-9268-6/22/07.

<https://doi.org/10.1145/3520304.3529056>

method exhibited significant difficulty translating to accuracy in classification tasks [8].

The most recent research suggests, however, that the gap between GP and ML in tasks of classification has started to shrink. Some GP-based classifiers have been shown to be competitive with other ML methods [14]. Further, methods designed for GP-based regression were shown to be successful also in classification tasks [12, 16]. Additionally, new promising methods have been proposed [3, 24–26]. All the aforementioned reasons lead us to formulate the following questions: 1) Are there any obstacles that prevent translating competitive performance of GP-based methods from regression into classification problems? 2) Why are there so many publicly available symbolic regression methods and just a handful of symbolic classification ones? 3) Why has there been few studies focusing on classification using GP? And finally, 4) Why are there no systematic studies that would attempt to compare GP classifiers to ML methods? Our paper attempts to close this gap. Herein, we compare performance of evolutionary classification approaches on the DIGEN benchmark. We also explore if symbolic GP-based approaches can match the performance of some of the leading classifiers in the general ML field.

2 METHODS

In this study, we cover four modern GP-based classifiers for which the implementation has been released as open source: *gplearn*, M3GP, M4GP and FEAT. We note that the selection of open-source implementations of many GP-based classifiers is limited. For benchmarking we use collection of 40 different datasets included in DIGEN [22]. Heavily tuned state-of-the-art ML classifiers serve there as baseline models (XGBoost, LightGBM, Gradient Boosted Trees, Random Forest, Decision Tree, SVC, Logistic Regression and K-nearest neighbors).

2.1 DIGEN

Diverse and generative ML benchmark (DIGEN) is a collection of 40 problems created using multi-stage evolutionary optimization process, in which the goal was to diversify performance of eight leading ML methods [22]. DIGEN ground truth is built in form of GP trees built with simple mathematical and logical operators. The generative functions as well as specific random seeds used for the experiments are presented in Supplementary Material. The baseline performance on each of the datasets of eight leading ML algorithms was included in the benchmark.

2.2 Evolutionary and GP-based classifiers

In this paper we compared performance of four GP-based classifiers.

M3GP. M3GP's fundamental idea is to find a transformation of data so that each class can be uniquely clustered [19]. To achieve that, the algorithm evolves a population of trees used to map data into d -dimensional space (each branch corresponds to a transformation into a single dimension).

M4GP. M4GP [14, 15] uses the idea of M3GP algorithms to represent features and classify samples in the new feature space, but it comes with two improvements over its predecessors. The standard tree-based representation of equations was replaced by stack-based

that allows for multiple outputs out-of-the-box. Secondly, the algorithm incorporates lexicase selection [10], a parent selection pattern that assures that a population is highly diverse throughout the evolution.

gplearn. A *gplearn* library for GP is implemented in Python and offers a scikit-learn compatible API [28]. It contains a rich set of tools needed for performing symbolic regression tasks. Also, binary classification is possible through the sigmoid function that converts regression output into the probability of being an instance of a class.

FEAT. Feature Engineering Automation Tool (FEAT) [12, 16] takes advantage of multidimensional GP similarly to M3GP. It operates on sets of programs that constitute the input for a logistic regression model in the case of binary classification. In addition, it incorporates semantic mutation and crossover operators [11] as well as gradient descent to learn weights within each feature.

2.3 Experiment design

The settings of the methods used in this benchmarking study on DIGEN collection are presented in Table 1. The settings were chosen based on the preferable settings from the papers or documentation, and expert knowledge.

Table 1: Settings of evolutionary methods

GP-based classifiers			
M3GP	'max_generation'	discrete	[10, 200] by 10
	'population_size'	discrete	[50, 500] by 10
	'tournament_size'	discrete	[5, 20]
	'limit_depth'	discrete	[2, 10]
M4GP	'g'	discrete	[10, 200] by 10
	'popsize'	discrete	[50, 500] by 10
	'max_len'	discrete	[10, 50] by 5
	'rt_cross'	discrete	[0.1, 0.9] by 0.1
	'rt_mut'	discrete	1 - rt_cross
	'fit_type'	categorical	'F1', 'F1W'
	'selection'	categorical	'lexicase', 'tournament'
gplearn	'generations'	discrete	[10, 200] by 10
	'population_size'	discrete	[50, 500] by 10
	'tournament_size'	discrete	[10, 25]
	'p_crossover'	discrete	[0.1, 0.9] by 0.1
FEAT	'gens'	discrete	[10, 200] by 10
	'pop_size'	discrete	[50, 500] by 10
	'max_depth'	discrete	[2, 10]
	'max_dim'	constant	10
	'sel'	constant	'lexicase'
	'max_stall'	constant	50
	'ml'	constant	'L1_LR'
	'fb'	constant	1

In comparison to the prior paper by Orzechowski and Moore [22], where ML methods were allowed to use 200 different evaluations for tuning (each with separate set of parameters) regardless of their execution time, in our comparison of evolutionary/GP-based classifiers those assumptions had to be adjusted.

In contrast to a popular symbolic regression benchmark *SRBench* by La Cava et. al [13], we have opted not to fix the number of evaluations allowed per each GP method. The main reason for that is that in our belief that allowing any GP methods to run for say 100,000 or 1,000,000 evaluations (generations times population size) could result in an unfair time advantage over classic ML methods, which are allowed for only 200 evaluations. Moreover, the datasets in DIGEN may not be complex enough to warrant such a high number of evaluations. Additionally, we have noticed that some of

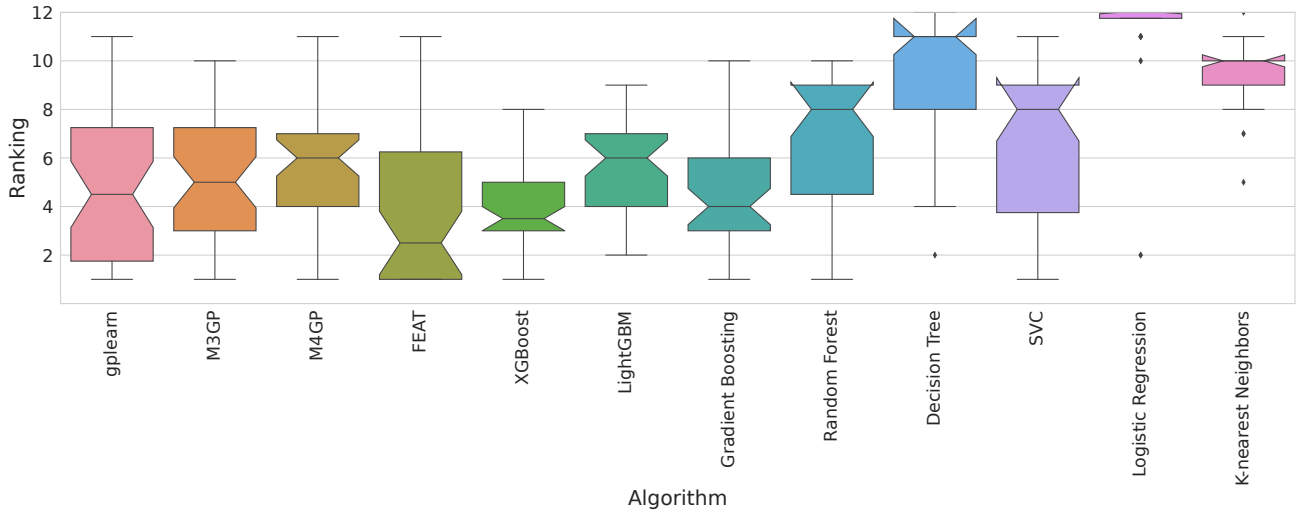


Figure 1: Ranking of the methods on the benchmark. Lower is better. The results were rounded to 3 decimal digits.

GP methods, namely *gplearn*, does not store by default a so-called hall of fame, or best solutions across all the evaluations, and instead returns the best individual from the last generation¹. Thus, we have opted to randomly determine up to 200 different settings of population sizes and generations.

Rather than setting iteration limits, we have instead enforced timeout constraints. We allowed GP-based methods not to launch any new evaluations after exceeding the time limit. Thus, the methods were allowed to run with a single core for up to 100,000 seconds (around 27.7 hrs) per dataset on an Intel(R) Xeon(R) CPU E5-2690 v3 @ 2.60GHz with 10GB of RAM.

The settings of GP methods were chosen randomly based on the values presented in Table 1. All constraints of the methods were passed to DIGEN benchmark, which is Optuna, a hyper-parameter optimization framework [1]. Optuna controls the runs, allows to specify number of trials, timeouts, as well as application of different distributions of the input parameters for a given method (including discrete, float, categorical, uniform, loguniform distributions).

As it was previously mentioned, the GP-based algorithms were allowed to launch experiments until the 100,000 second threshold was reached. This time was deemed as sufficient to converge and solve fairly simple classification tasks with datasets having 10 columns and 1000 rows. Over this time, the choice of the parameter settings was passed to Optuna.

3 RESULTS

GP-based methods performed surprisingly well on the DIGEN benchmark with multiple classifications scenarios. They were certainly very competitive with ML algorithms, including those leading ones (XGBoots, LightGBM or Gradient Boosting). We observed these performance results despite most methods using fewer than 100k evaluations due to the time constraint. According to median

ranks of the methods (see Fig. 1), FEAT was overall the best classifier in the study, followed by XGBoost and Gradient Boosting. Two other GP-based methods, *gplearn* and M3GP followed with slightly worse overall performance. The results of the remaining GP-based method, namely M4GP, were competitive with the next ML method - LightGBM. The remaining ML methods were clearly outperformed, with Random Forest and SVC performing better than k-nearest neighbors and Decision Trees. Logistic regression clearly trailed in the comparison.

It is worth mentioning here that GP-based method use different programming languages. For example, *gplearn* and M3GP are implemented in Python, whereas FEAT use highly efficient C++ as their backend, Python is only used as a wrapper. The number of actually executed runs per each of the GP-methods visibly differed between the methods. M4GP and *gplearn* almost used all potentially available runs and ran on 192.4 and 198.6 trials on average, while FEAT was able to launch only 20.7 optimization tasks on average across all the datasets. Surprisingly, M3GP managed to launch only 3.1 trials on average, which is far less compared to all of the competitors. It is clear from the results that the number of trials doesn't constitute better performance, as longer runs with larger generations and population size might have lead to better performances. The number of actual trials is however influenced by both the complexity of the method as well as its implementation.

In our tests, FEAT was the only method that ran out of 10GB memory limit, and did it on two datasets (*digen27* and *digen36*) and was halted prematurely. Interestingly, the runs with higher memory limits (50GB) finished successfully, but provided different results despite setting *seed/random_state* variables correctly in order to provide reproducibility of the experiment. This is likely due to the multi-threaded and randomized nature of these algorithms.

4 DISCUSSION

We gather from the results that significant progress has been made in the area of evolutionary classification. So why does this research

¹<https://github.com/trevorstevens/gplearn/issues/107>

remain unnoticed or adapted more widely? In our opinion there are five major factors that contribute to this current state. The first one is **publishing open source**. Multiple evolutionary methods do not provide publicly available implementation that could be used, adapted and/or extended. The closed-source nature prevents the methods from being benchmarked or applied. The second is **speed**. Some implementations of the methods are slow (e.g. implemented in high-level programming languages) and/or not scalable (without multi-core support). The choice of programming language and effortless installation could also play an important role. Additionally, existing methods do not offer sufficient **wrappers, integrations and interfaces** to popular data science libraries, such as scikit-learn [23], or a standardized way of loading data from CSV files or pandas DataFrames [18]. This makes some of the methods not ready out-of-the-box for modern ML. Similarly, the availability of decent **documentation and tutorials** allow users to have hands-on experience using the software. Finally, there has been **no established benchmark** that would tackle comparison of GP-based classifiers against leading ML competitors.

Overall, in the coming years we hope to witness a surge of GP and evolutionary-based classifiers published open source, with scikit-learn wrappers that would offer high scalability. We believe this is the right direction to further strengthen ties between evolutionary and data science communities and attract wider public interest.

SUPPLEMENTARY MATERIALS

The materials and scripts are publicly available at the following website: <https://github.com/athril/gpbench>.

ACKNOWLEDGMENTS

This work was supported in part by the European Union's Horizon 2020 research and innovation programme under grant agreement Sano No. 857533 and the International Research Agendas programme of the Foundation for Polish Science, co-financed by the EU under the European Regional Development Fund, NIH grants R01AG066833 and R01LM010098, and PL-Grid infrastructure. William La Cava was supported by the National Library of Medicine and National Institutes of Health under award R00LM012926.

REFERENCES

- [1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. 2019. Optuna: A Next-generation Hyperparameter Optimization Framework. *arXiv:1907.10902* [cs.LG]
- [2] Bernd Bischl, Giuseppe Casalicchio, Matthias Feurer, Frank Hutter, Michel Lang, Rafael G Mantovani, Jan N van Rijn, and Joaquin Vanschoren. 2017. OpenML benchmarking suites and the OpenML100. *stat* 1050 (2017), 11.
- [3] Karina Brotto Rebuli and Leonardo Vanneschi. 2022. An Empirical Study of Progressive Insular Cooperative GP. *SN Computer Science* 3 (03 2022). <https://doi.org/10.1007/s42979-021-00998-7>
- [4] Mauro Castelli, Sara Silva, Leonardo Vanneschi, Ana Cabral, Maria J Vasconcelos, Luis Catarino, and João MB Carreiras. 2013. Land cover/land use multiclass classification using GP with geometric semantic operators. In *European Conference on the Applications of Evolutionary Computation*. Springer, 334–343.
- [5] Pedro G. Espejo, Sebastián Ventura, and Francisco Herrera. 2010. A Survey on the Application of Genetic Programming to Classification. *Trans. Sys. Man Cyber Part C* 40, 2 (mar 2010), 121–144.
- [6] Vijay Ingalalli, Sara Silva, Mauro Castelli, and Leonardo Vanneschi. 2014. A multi-dimensional genetic programming approach for multi-class classification problems. In *European Conference on Genetic Programming*. Springer, 48–60.
- [7] Hajira Jabeen and Abdul Baig. 2010. Review of Classification Using Genetic Programming. *International Journal of Engineering Science and Technology* 2 (02 2010).
- [8] Michael F Korns. 2018. An Evolutionary Algorithm for Big Data Multi-Class Classification Problems. In *Genetic Programming Theory and Practice XIV*. Springer, 165–178.
- [9] Alex Krizhevsky and Geoffrey Hinton. 2009. *Learning multiple layers of features from tiny images*. Technical Report 0. University of Toronto, Toronto, Ontario.
- [10] William La Cava, Thomas Helmuth, Lee Spector, and Jason H. Moore. 2019. A Probabilistic and Multi-Objective Analysis of Lexicase Selection and ϵ -Lexicase Selection. *Evolutionary Computation* 27, 3 (09 2019), 377–402. https://doi.org/10.1162/evco_a_00224 *arXiv:https://direct.mit.edu/evco/article-pdf/27/3/377/1858632/evco_a_00224.pdf*
- [11] William La Cava and Jason H. Moore. 2019. Semantic Variation Operators for Multidimensional Genetic Programming. In *Proceedings of the 2019 Genetic and Evolutionary Computation Conference (GECCO '19)*. ACM, Prague, Czech Republic. <https://doi.org/10.1145/3321707.3321776> *arXiv:1904.08577*
- [12] William La Cava and Jason H. Moore. 2020. Genetic Programming Approaches to Learning Fair Classifiers. In *Proceedings of the 2020 Genetic and Evolutionary Computation Conference (Cancún, Mexico) (GECCO '20)*. Association for Computing Machinery, New York, NY, USA, 967–975. <https://doi.org/10.1145/3377930.3390157>
- [13] William La Cava, Patryk Orzechowski, Bogdan Burlacu, Fabricio de Franca, Marco Virgolin, Ying Jin, Michael Komenda, and Jason Moore. 2021. Contemporary Symbolic Regression Methods and Their Relative Performance. *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1* (Dec. 2021). *arXiv:2107.14351*.
- [14] William La Cava, Sara Silva, Kourosh Danaei, Lee Spector, Leonardo Vanneschi, and Jason H Moore. 2019. Multidimensional genetic programming for multiclass classification. *Swarm and evolutionary computation* 44 (2019), 260–272.
- [15] William La Cava, Sara Silva, Leonardo Vanneschi, Lee Spector, and Jason Moore. 2017. Genetic programming representations for multi-dimensional feature learning in biomedical classification. In *European Conference on the Applications of Evolutionary Computation*. Springer, 158–173.
- [16] William La Cava, Tilak Raj Singh, James Taggart, Srinivas Suri, and Jason H. Moore. 2019. Learning concise representations for regression by evolving networks of trees. In *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/1807.00981>
- [17] M. Lichman. 2013. UCI Machine Learning Repository. <http://archive.ics.uci.edu/ml>
- [18] Wes McKinney et al. 2010. Data structures for statistical computing in python. In *Proceedings of the 9th Python in Science Conference*, Vol. 445. Austin, TX, 51–56.
- [19] Luis Muñoz, Sara Silva, and Leonardo Trujillo. 2015. M3gp-multiclass classification with gp. In *European Conference on Genetic Programming*. Springer, 78–91.
- [20] Randal S Olson, William La Cava, Patryk Orzechowski, Ryan J Urbanowicz, and Jason H Moore. 2017. PMLB: a large benchmark suite for machine learning evaluation and comparison. *BioData mining* 10, 1 (2017), 1–13.
- [21] Patryk Orzechowski, William La Cava, and Jason H. Moore. 2018. Where Are We Now? A Large Benchmark Study of Recent Symbolic Regression Methods. In *Proceedings of the Genetic and Evolutionary Computation Conference (Kyoto, Japan) (GECCO '18)*. Association for Computing Machinery, New York, NY, USA, 1183–1190. <https://doi.org/10.1145/3205455.3205539>
- [22] Patryk Orzechowski and Jason H Moore. 2021. Generative and reproducible benchmarks for comprehensive evaluation of machine learning classifiers. *arXiv preprint arXiv:2107.06475* (2021).
- [23] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.
- [24] Wenbin Pei, Bing Xue, Lin Shang, and Mengjie Zhang. 2021. High-dimensional Unbalanced Binary Classification by Genetic Programming with Multi-criterion Fitness Evaluation and Selection. *Evolutionary Computation* (12 2021), 1–26. https://doi.org/10.1162/evco_a_00304 *arXiv:https://direct.mit.edu/evco/article-pdf/doi/10.1162/evco_a_00304/1976864/evco_a_00304.pdf*
- [25] Stefano Ruberto, Valerio Terragni, and Jason H. Moore. 2021. *Towards Effective GP Multi-Class Classification Based on Dynamic Targets*. Association for Computing Machinery, New York, NY, USA, 812–821. <https://doi.org/10.1145/3449639.3459324>
- [26] Leo Willyanto Santoso, Bhopendra Singh, S. Suman Rajest, R. Regin, and Karar Hameed Kadhim. 2020. A Genetic Programming Approach to Binary Classification Problem. *EAI Endorsed Transactions on Energy Web* 8, 31 (7 2020). <https://doi.org/10.4108/eai.13-7-2018.165523>
- [27] Sara Silva and Ernesto Costa. 2009. Dynamic limits for bloat control in genetic programming and a review of past and current bloat theories. *Genetic Programming and Evolvable Machines* 10, 2 (2009), 141–179.
- [28] Trevor Stephens. 2016. *Genetic Programming in Python, with a scikit-learn inspired API: gplearn*.